Assignment-4 (Practice Problems)

Git Commit Requirements

Your Git history should show the progression of your work, not just the final solution.

What is required

- Make a commit after each meaningful change or milestone (creating a class header or source file, implementing a substantial class method, completing one part of the assignment, adding a new feature, or fixing a significant bug).
- Each commit message must briefly explain what you implemented, fixed, or changed.
- Commit regularly while working rather than committing the entire assignment at the end.
- Create commits as you work. Do not complete the assignment first and then divide the finished code into several commits afterward.
- You do not need to push after every commit. You may commit your changes locally and push all commits later.

Good commit examples

- `Implemented input validation in main function`
- `Fixed incorrect calculation in method getValue`
- `Updated Car class constructor`
- `Corrected output formatting for method print`
- `Implemented toString method in House.cpp`

Not meaningful

- `commit 1`
- `update`
- `test`
- `done`
- `final`
- Multiple commits that contain no meaningful change

Grading
Git-related deductions are not cumulative. The highest applicable deduction will be used.

- **Up to -20% — Poor commit quality**: You made multiple commits, but the messages are vague, repetitive, or do not clearly describe your changes, or your commits do not consistently represent meaningful progress.

- **Up to −40% — Insufficient development history:** Your Git history does not reasonably demonstrate incremental development. Examples include submitting most or all of the assignment in one or a few very large commits, making nearly all significant commits immediately before the deadline, creating several commits only after the code was already completed, or committing a large amount of new code only a few minutes after the previous commit.

A large number of commits by itself does not satisfy this requirement. Your commits should reflect actual, meaningful progress in your code.

Problem-1 (11 points):

Create a program to shuffle and deal a deck of cards. The program should consist of class Card and class Deck.

Class Card should provide:

1. An integer data field (private) named face.
2. An integer data field (private) named suit.
3. Two static vectors of strings (private) representing the faces and suits.

```
static std::vector<std::string> faceNames;
static std::vector<std::string> suitNames;
```

4. Initialize static members outside of the class in Card.cpp.

```
std::vector<std::string> Card::faceNames { "A", "2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K"};
std::vector<std::string> Card::suitNames {"♥", "♦", "♣", "♠"};
```

5. The accessor functions for face and suit, defined as constant member functions.
6. A constructor that receives two ints representing the face and suit to initialize the data members. The face value should be in the range [0; 12]. The suit value should be in the range from [0; 3].

```
Card(int face_num, int suit_num);
```

7. A toString constant function that returns the Card as a string in the form “face suit.”

```
J♣
```

Class Deck should contain:

1. A vector of Cards (private) named deck to store the Cards.
2. An integer (private) named current_card representing an index of the next card to be dealt.
3. An integer (private) named shuffle_seed representing the seed of random numbers.
4. A constructor that takes a seed as an argument and initializes the Cards in the deck using push_back(). The constructor should initialize the current_card property with zero.
5. A void shuffle function that shuffles the Cards in the deck. The shuffle algorithm should iterate through the vector of Cards. For each Card, randomly select another Card in the deck and swap the two Cards. Set current_card to zero at the end.

   To generate random numbers:

```
// Initialize a random number engine with a seed
std::mt19937 eng(shuffle_seed);
//Define a distribution
std::uniform_int_distribution<int> dist(0, 51);
// Generate a random value
int random_val = dist(eng);
```

6. A dealCard function that returns the current card from the deck and increments current_card counter.
7. A moreCards constant function that returns a bool value indicating whether there are more Cards to deal. There are 52 cards in the deck.

Problem-2 (12 points):

Implement class Hand, that contains:

1. A vector of Cards (private) named hand that stores player's cards. The hand size is always 5 cards.
2. A void method takeCards() that takes a reference to a Deck object. The method discards all current cards in hand (you may use clear() method of class vector) and then takes 5 cards from the given deck. Before taking each card, the method must verify that the deck is not empty.
3. A boolean constant method isFullHand() that returns true if the hand has exactly 5 cards and false otherwise.

4. A void constant method print() that prints all cards in hand. (use toString() card class method)
5. A pair constant function that returns a bool value indicating whether the hand contains a pair.
6. A twoPair constant function that returns a bool value indicating whether the hand contains two pairs.
7. A threeOfAKind constant function that returns a bool value indicating whether the hand contains three of a kind. (e.g., three jacks)

Sample run (not full):

```
J♣
Enter seed: 1

******************BEFORE SHUFFLING******************
A♥ A♦ A♣ A♠ 2♥ 2♦ 2♣ 2♠ 3♥ 3♦ 3♣ 3♠ 4♥ 4♦ 4♣ 4♠ 5♥ 5♦ 5♣ 5♠ 6♥ 6♦ 6♣ 6♠ 7♥ 7♦ 7♣ 7♠ 8♥ 8♦ 8♣ 8♠ 9♥ 9
♦ 9♣ 9♠ 10♥ 10♦ 10♣ 10♠ J♥ J♦ J♣ J♠ Q♥ Q♦ Q♣ Q♠ K♥ K♦ K♣ K♠
******************AFTER SHUFFLING******************
2♥ 5♣ 10♦ K♥ 7♣ 2♣ 4♠ 10♥ A♦ 3♦ 10♣ 6♠ 4♥ 3♠ 5♦ 9♣ 5♠ A♠ 8♥ Q♥ A♥ 9♥ 6♣ 9♦ 6♦ 7♠ 8♠ 8♦ K♠ 8♣ 2♦ Q♠ 4
♦ 7♦ 6♥ 9♠ Q♣ 10♠ 7♥ J♥ J♦ J♠ K♣ 2♠ K♦ A♣ J♣ Q♦ 3♣ 5♥ 3♥ 4♣
```

```
------------------
7♣ 8♥ 10♠ 3♣ 8♠
Hand contains a pair
------------------
------------------
4♠ 9♣ Q♣ 5♣ 3♦
------------------
------------------
7♥ 9♦ 4♥ 6♠ A♠
------------------
------------------
6♥ Q♥ K♥ K♠ K♦
Hand contains three of a kind
------------------
```

```
------------------
8♣ Q♦ 2♦ 5♦ 2♣
Hand contains a pair
------------------
------------------
4♠ A♣ 6♠ 8♦ 2♥
------------------
------------------
9♠ K♣ J♦ 9♥ J♠
Hand contains two pairs
------------------
```

Testing:

Run main.cpp (`gcc *.cpp`) to test your code. You can comment some part out to test a specific class.

Submission:

Once your program works correctly and the output format is correct, submit your work using the following Git commands:

```
git add <file_name>
git commit -m "Commit message"
git push
```